

LearnOpenCV

by B:G VISION

(<https://learnopencv.com>)

Home(<https://Learnopencv.com/>)

Getting Started ∨ (<https://Learnopencv.com/Category/Introduction>)

OpenCV University ∨ (https://Opencv.org/University/?Utm_source=Lopcv&Utm_medium=M)

Free Courses ∨ (https://Opencv.org/University/Free-Courses/?Utm_source=Lopcv&Utm_medium=Menu&Utm_medium=Courses)

AI Consulting (<https://Learnopencv.com/AI-Consulting>)

About(<https://Learnopencv.com/About/>)

YouTube

(https://Bit.ly/YouTube_LearnOpenCV)

MedSAM2 Explained: One Prompt to Segment Anything in Medical Imaging

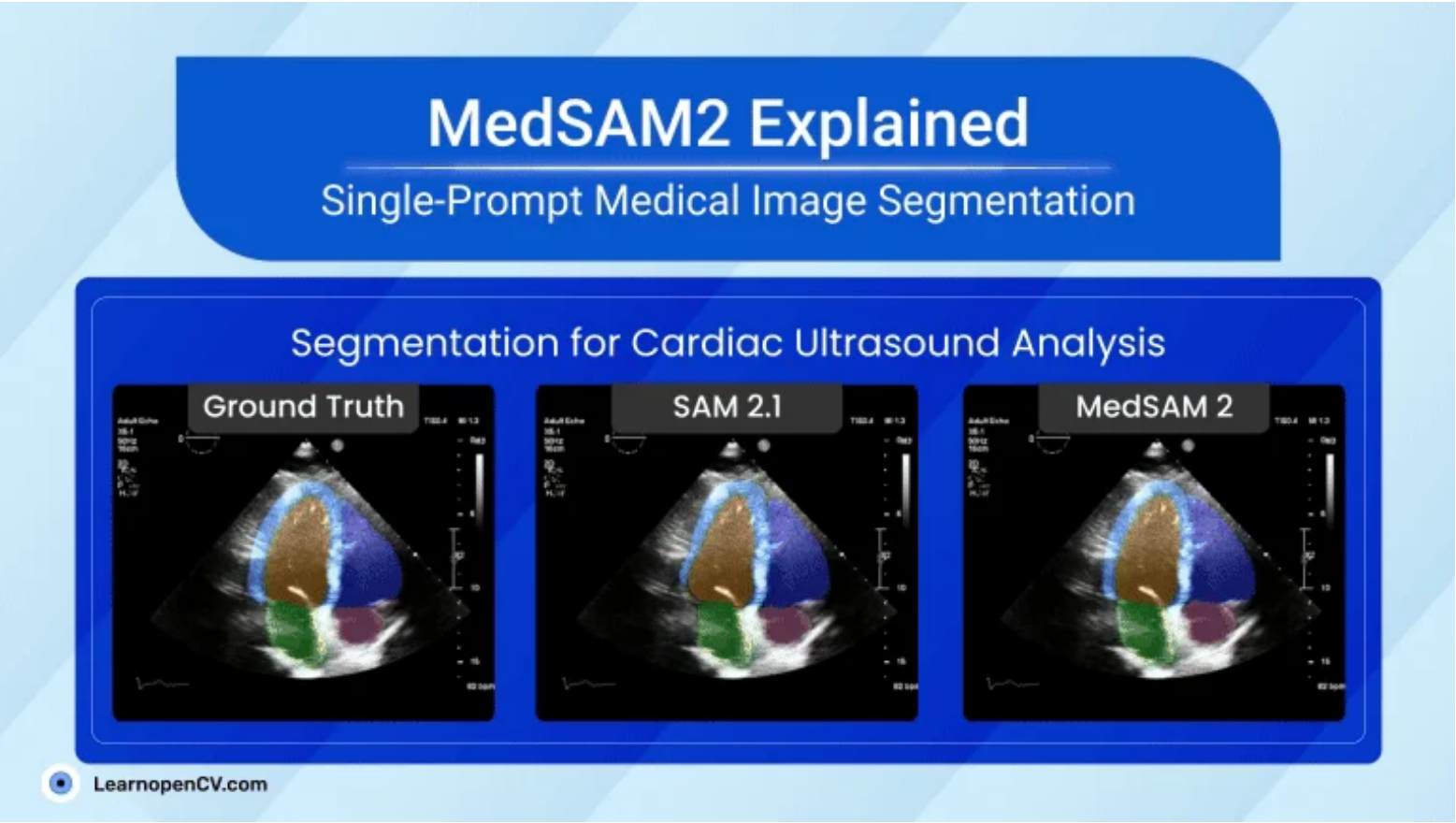
 [Ankan Ghosh \(https://learnopencv.com/author/ankan/\)](https://learnopencv.com/author/ankan/)

APRIL 29, 2025

[3D Computer Vision \(https://learnopencv.com/category/3d-computer-vision/\)](https://learnopencv.com/category/3d-computer-vision/) [Computer Vision \(https://learnopencv.com/category/computer-vision/\)](https://learnopencv.com/category/computer-vision/) [Image Segmentation \(https://learnopencv.com/category/image-segmentation/\)](https://learnopencv.com/category/image-segmentation/)

Evening rush hour, a small country hospital, the scan room hums like an old fridge while a tired doctor checks the last patient of the day – a farmer who fell off his tractor and now finds it hard to breathe. The CT machine spins, making a pile of more than a thousand grey pictures. Every minute matters: is there a bruised lung, hidden bleeding, or something worse? Now, the doctor would click through each slice, drawing lines around the lungs and ribs, and decide what to do next with the farmer, a job that could take an hour or even more! However, not anymore, **MedSAM2** is here to help. But How?

The doctor provides all the images and draws a quick box over the chest, and MedSAM2 detects and segments all the parts clearly throughout the images. In seconds, it colors the lungs green and the heart blue and shows a dark red spot where blood is collecting. The doctor calls the surgery team immediately, and the farmer heads to the operating room before the CT table has even slid out.



That tiny story shows a significant shift in AI in Healthcare. New **MedSAM2**, developed by WangLab (the developers of **MedSAM**), can outline almost any part of any 3D scan or medical video with just one click. Tasks that once took hours now take minutes, providing doctors with faster answers and patients with quicker assistance.

In this article, we will explore:

- **Current trends in medical-AI research and why segmentation sits centre-stage.**
- **What medical imaging really is, and how MedSAM2 will help here.**
- **A friendly tour of the SAM – MedSAM – SAM2 – MedSAM2 architecture journey.**
- **Code workflow and inference results.**
- **A quick recap of the article.**
- **All the modified Notebooks are given in the [Download Code Button](#).**

Now, that's exciting, right? So grab a cup of coffee, and let's dive in!

Table of Contents

1. Current Trends in Medical AI
2. What is Medical Imaging? How MedSAM2 can help!
3. Introduction to MedSAM2
4. Dataset Preparation and Training
5. Code Pipeline
6. TL;DR
7. Conclusion
8. References

Current Trends in Medical AI

Medical imaging is at the heart of AI's healthcare boom, and the numbers tell a thriller-style story. In just twelve months, the global AI market in medical imaging jumped from approximately **USD 1 billion in 2023 to USD 1.28 billion in 2024**; analysts now predict the sector will reach anywhere between **USD 14 billion and USD 24 billion by the early 2030s**, implying a compound annual growth rate of over 30%.

(<https://learnopencv.com/wp-content/uploads/2025/04/medical-image-segmentation-market-value-in-healthcare.png>),

Rise of Foundation models

Why that pace? One word: **foundation models**. Just as **GPT-style language models** are rewriting NLP, vision language models (like **SAM**) are revolutionizing Vision. A recent arXiv survey reveals more than **40 new “segment-anything” spin-offs for medical scans** in the past 18 months alone, with conference tracks and entire **CVPR** (<http://arxivfmv-cvpr25workshop.github.io>) workshops emerging to cover them.

Segmentation is the new gateway for Healthcare

Hospitals often adopt AI first for *segmentation* because it's concrete, auditable, and meshes with existing workflows. Faster, sharper borders mean:

- **Fewer surgical surprises** – oncology teams can sculpt radiation beams with sub-millimetre precision.
- **Quicker reads in busy ERs** – one bounding box on a CT slice, and a model like **MedSAM2** outlines the spleen or a bleeding lesion in under a second.
- **Better downstream AI** – clean masks feed volumetric tumour-growth models or 3-D printing pipelines without manual cleanup.

What is Medical Imaging? How MedSAM2 can help!

Medical imaging is just **taking pictures of the inside of the body**. X-rays scan bones, CT slices you like a loaf of bread, MRI looks at soft tissue, PET lights up metabolism, and ultrasound films organs in motion. All those pixels let doctors see what the hands can't feel.

Massive Workload – A single abdominal CT scan can produce over **1,000 slices**; an echocardiogram can record the heart beating 60 times a second. Colouring every slice by hand is like painting each frame of an entire Pixar film.

MedSAM2 as a Solution – Drop one click or box, and the model fills in the exact border. From there, doctors can measure tumour volume, steer a radiation beam, or track disease over months.

Four major areas MedSAM2 solves –

Imaging Task	Old Pain	How MedSAM2 fixes it
CT Segmentation	Lesion borders blur into organ tissue.	Hits DSC 0.95 for liver, 0.68 for lesions – about +5 pp over nnU-Net.
MRI Segmentation	Variable contrast hides tumours.	Higher median DSC on brain & liver tasks; easily outlines tricky cervical-cancer margins.
PET Lesion Detection	Noise and benign uptake cause false positives.	DSC $\approx$ 0.68 , center-of-mass error $\sim$ 2 mm when fusing PET+CT.
Video Segmentation (US & Endoscopy)	Motion blur and speckle kill accuracy.	Polyp DSC 91.3 % ; heart-chamber edges stay crisp in real-time echo.

Now, let’s dive into the Model itself and see how the MedSAM2 was created.

Introduction to MedSAM2

Every major breakthrough often builds upon a strong foundation, and MedSAM2 is no exception. Its development traces back to one of the most influential models in recent computer vision research: Meta AI’s Segment Anything Model 2 (SAM2). SAM demonstrated that a single, prompt-driven network could accurately segment virtually any object within natural images, from animals and vehicles to everyday objects, using minimal user input, such as a single click.

SAM – MedSAM – SAM2 – MedSAM2

SAM (Apr 2023)

Meta AI’s first **Segment Anything Model** (<https://learnopencv.com/segment-anything/>) proved that a single click could outline almost any object in a photo. It learned from a billion natural-image masks and turned segmentation into a point-and-paint trick.

MedSAM (Apr 2023)

Researchers at WangLab asked, “What if we fine-tune SAM on hospital scans?” They fine-tuned the model on 1.5 million CT, MRI, and X-ray slices. **MedSAM** (<https://github.com/bowang-lab/MedSAM>) could now color livers and tumors much like SAM colors coffee cups, but only slice by slice, still in pure 2D.

SAM2 (Nov 2024)

Meta’s upgrade retained the click-anywhere spirit but replaced the heavier **Hiera** transformer backbone with a lighter one, introducing a tidier prompt and mask head that made the network faster and less memory-intensive. **SAM2** (<https://learnopencv.com/sam-2/>) still thinks in flat

frames; it has no built-in way to remember what came before or after a frame in a stack or a clip.

MedSAM2 (Apr 2025)

The newest step combines the best of both worlds. It retains SAM2's speedy Hiera core and adds a small memory-attention block, allowing every CT slice or video frame to examine its eight neighbors. Trained on a 10-modality mix (≈ 450 k 3-D volumes plus 76 k video frames), MedSAM 2 understands whole volumes and live ultrasound in one go while still running quickly on a regular workstation GPU.

In one sentence: SAM proved the idea, MedSAM made it medical, SAM 2 made it fast, and MedSAM 2 makes it work in full 3-D and real-time video by giving the network a short-term memory.

How MedSAM 2 Works Inside

The diagram illustrates the MedSAM2 architecture, showing how it integrates a memory-attention block with the Hiera core. It details the flow of information from input frames through the attention mechanism to the final segmentation output.

Smaller, faster Input

Images are downsized to 512×512 instead of 1024×1024 , which matches most medical slices and halves the computational load.

The image encoder is a four-stage Hiera vision transformer. Extra global-attention blocks allow far-apart pixels, such as the two tips of a long vessel, to share information.

Above the backbone sit **four transformer layers** that read from a small memory bank holding the previous slices or frames. Rotary position embeddings inform the model precisely where each feature is located in space or time, ensuring that neighboring slices align cleanly and moving borders do not jitter.

A tiny prompt encoder changes a user's box, point, or scribble into vector embeddings the network can follow, like dropping a pin on a map.

The mask decoder combines the prompt clues with multi-scale features, generates a 128×128 mask, and then scales it back to full size. One set of weights now outlines livers in CT stacks *and* beating ventricles in echo videos, guided by nothing more than a quick click.

Our courses cover Image Segmentation, Fundamentals of Computer Vision, and Deep Learning in depth. To get started, just click below on any of our free Bootcamps!

 100K+ Learners  3 Hours of Learning	 15K+ Learners  3 Hours of Learning	 10K+ Learners  8 Hours of Learning
Join Free OpenCV Bootcamp https://opencv.org/university/free-opencv-course/?utm_source=locv&utm_medium=midblog&utm_campaign=explained-one-prompt-to-segment-anything-in-medical-imaging	Join Free TensorFlow Bootcamp https://opencv.org/university/free-tensorflow-keras-course/?utm_source=locv&utm_medium=midblog&utm_campaign=explained-one-prompt-to-segment-anything-in-medical-imaging	Join Free PyTorch Bootcamp https://opencv.org/university/free-pytorch-course/?utm_source=locv&utm_medium=midblog&utm_campaign=explained-one-prompt-to-segment-anything-in-medical-imaging

[View all AI Free Courses](https://opencv.org/university/free-courses/?utm_source=lopcv&utm_medium=blog) >
(https://opencv.org/university/free-courses/?utm_source=lopcv&utm_medium=blog)

Dataset Preparation and Training

Dataset Preparation for MedSAM2

To teach MedSAM 2, the team first built one very large set of scans. It contains **approximately 450,000 full 3D studies** from reputable sources, including LiTS, BraTS, KiTS, FLARE, TotalSegmentator, AutoPET, and several hospital archives. These cover CT, MRI, and PET, so the model is exposed to a wide range of tissue types, imaging machines, and patient populations. They also added approximately 76,000 video frames from echo and endoscopy sets, such as EchoNet-Dynamic and Hyper-Kvasir, giving the network a sense of moving anatomy.

Modality	Key public sets used	What they add to the mix
CT 3-D	LiTS (liver + tumours), KiTS19 (kidneys), FLARE22 (multi-organ), TotalSegmentator (104 structures)	High-contrast organs and diverse lesions
MRI 3-D	BraTS (brain tumours), ACDC (cardiac), CHAOS (abdominal organs), MSD #01–10	Soft-tissue edges, variable contrast
PET/CT	AutoPET, Head-Neck Cancer Seg, MICCAI TOTALSEG-PET subset	Hot vs. cold lesion boundaries
Ultrasound video	EchoNet-Dynamic & CAMUS (heart), CLUST (fetal),	Motion + speckle noise
Endoscopy video	Hyper-Kvasir, Kvasir-SEG, PolypGen	Polyp shapes, lighting shifts

Every file went through the same cleaning steps:

- CT slices were resized to one-millimetre cubes and their grey values clipped to normal limits.
- MRI and ultrasound images were scaled so that dark and bright parts sit in a similar range.
- Video clips were cropped to 512×512 pixels and fixed at 30 frames per second.

Older models drew a quick “starter” mask (nnU-Net for 3-D scans, a small Mask R-CNN for video). Human reviewers then opened a web tool that highlighted the shaky parts of each mask in red; they only had to tidy those spots.

 (https://learnopencv.com/wp-content/uploads/2025/04/medsam2-human-in-loop-annotation.png).

You may have read about **three extra datasets** in the MedSAM 2 paper: one for **CT lesions (DeepLesion)**, one for MRI liver tumours (**LLD-MMRI**), and one for echo videos (**RVENet**). These *did not* join the big training pool. The authors saved them for later experiments. First, they timed how long doctors needed to correct MedSAM 2's first-try masks on those scans and showed an **85-92% reduction in labor** compared with drawing by hand. Next, they performed a brief additional fine-tuning on the corrected masks, achieving an additional 3-6 Dice points. They also released a MedSAM2-annotated version of these datasets.

MedSAM2 – Training Tweaks

 **Click here to download the source code to this post**
One large mixed dataset – MedSAM 2 trained on ~~approximately 450,000~~ **76,000** labelled CT, MRI, and PET scans, as well as **76,000** labelled frames from ultrasound and endoscopy videos. Every file was resized to 512×512 (or 1 mm^3 voxels for 3D) and placed on the same grayscale scale, so the model never had to guess what “bright” or “dark” meant.

Fast “draft-and-fix” labels – Old networks (nnU-Net for 3D, a small Mask R-CNN for video) drew rough masks. Human reviewers only fixed the messy edges that the software coloured in red. This sped up labelling by a factor of ten.

Balanced batches – CT is common, PET is rare. During training, the loader quietly showed PET and endoscopy slices nearly twice as often and limited any single CT study to 5 % of a batch. That kept the model from turning into a “CT-only” expert.

Now, let's move on to the code!

Download Code To easily follow along this tutorial, please download code by clicking on the button below. It's FREE!

Download Code

Code Pipeline

Now we start playing with the model. For that, we need to clone the official MedSAM2 repository.

```
git clone https://github.com/bowang-lab/MedSAM2.git (https://github.com/bowang-lab/MedSAM2.git) && cd MedSAM2
```

We will then create a virtual environment to run all our experiments.

```
conda create -n medsam2 python=3.12 -y && conda activate medsam2
```

Then run this command to install all the requirements

```
pip install torch torchvision  
pip install -e ".[dev]"
```

One final Step, download the MedSAM2 checkpoints by running:

```
sh download.sh
```

We are now finished with the setup; let's proceed to the main code. We will explore two main applications here.

CT Lesion Segmentation

A **CT scan (Computed Tomography scan)** is a medical imaging technique that allows doctors to see inside the body with great detail. Unlike regular X-rays that show only a flat image, a CT scan takes many cross-sectional images (called slices) using rotating X-ray beams and a computer to create detailed pictures of bones, organs, blood vessels, and soft tissues. These images can then be viewed individually or reconstructed into a 3D model, giving doctors a clearer understanding of a patient's condition.

Here we will take 3D **CT scan** images of the **pelvis**, viewed in **axial slices** (horizontal cross-sections). And try to segment a lesion (probably a prostate tumor) with MedSAM2.

Section 1: Imports, Setup, and Argument Parsing

This initial part of the script sets up the necessary environment and obtains user input. It imports various Python libraries required for tasks like file handling (os, glob), numerical operations (numpy), data manipulation (pandas), image processing (PIL, SimpleITK, skimage), deep learning (torch), and plotting (matplotlib).

It also sets random seeds for PyTorch and NumPy to ensure that if you run the script multiple times with the same inputs, you get the same results (reproducibility). The `torch.set_..._precision(...)` line potentially optimizes matrix multiplication performance on specific newer GPUs.

```

1  from glob import glob
2  from tqdm import tqdm
3  import os
4  from os.path import join, basename
5  import re
6  import matplotlib.pyplot as plt
7  from collections import OrderedDict
8  import pandas as pd
9  import numpy as np
10 import argparse
11
12 from PIL import Image
13 import SimpleITK as sitk
14 import torch
15 import torch.multiprocessing as mp
16 from sam2.build_sam import build_sam2_video_predictor_npz # Specific predictor for NPZ input
17 import SimpleITK as sitk
18 from skimage import measure, morphology
19
20 # Set seeds for reproducibility and configure PyTorch performance
21 torch.set_float32_matmul_precision('high')
22 torch.manual_seed(2024)
23 torch.cuda.manual_seed(2024)
24 np.random.seed(2024)
25
26 # Initialize argument parser
27 parser = argparse.ArgumentParser()

```

Next, the script defines command-line arguments using `argparse`. This allows users to specify different settings when running the script without modifying the code itself.

```

1  parser.add_argument(
2      '--checkpoint',
3      default='checkpoints/MedSAM2_latest.pt',
4      help='checkpoint path',
5  )
6  parser.add_argument(
7      '--cfg',
8      default='configs/sam2.1_hiera_t512.yaml',
9      help='model config',
10 )
11
12 # more arguments here (skipped)
13
14 # Parse the arguments provided from the command line
15 args = parser.parse_args()
16 checkpoint = args.checkpoint
17 model_cfg = args.cfg
18 imgs_path = args.imgs_path
19 gts_path = args.gts_path
20 pred_save_dir = args.pred_save_dir
21 # Ensure the output directory exists, create if not
22 os.makedirs(pred_save_dir, exist_ok=True)
23 propagate_with_box = args.propagate_with_box

```

This section essentially prepares the script by importing necessary tools, setting up for consistent results, and defining how users can interact with it via the command line to control its behavior and input/output locations. The parsed arguments are then stored in variables for later use.

Section 2: Helper Functions

This section defines several utility functions that are used later in the script for tasks like image preprocessing, creating prompts, post-processing segmentations, and potentially visualization or evaluation.

```

1 def getLargestCC(segmentation):
2     # Finds connected components in the binary segmentation mask
3     labels = measure.label(segmentation)
4     # Check if there are any foreground labels (labels > 0)
5     if labels. ( ) == 0: # Handle empty segmentation
6         return segmentation # Return the empty segmentation as is
7     # Count occurrences of each label (excluding background label 0)
8     # Find the label with the largest count (largest connected component)
9     largestCC = labels == np.argmax(np.bincount(labels.flat)[1:])+1
10    # Return a mask containing only the largest connected component
11    return largestCC

```

getLargestCC(...): This function takes a binary segmentation mask (an array where pixels belonging to the segmented **object are 1 and background is 0**) and performs post-processing. It identifies all separate “blobs” or connected regions of foreground pixels. It then finds the largest blob (the one with the most pixels) and returns a new mask containing only this largest region.

```

1 def dice_multi_class(preds, targets):
2     smooth = 1.0
3     assert preds.shape == targets.shape
4     # Find unique foreground labels in the target mask
5     labels = np.unique(targets)[1:]
6     dices = []
7     for label in labels:
8         # Create binary masks for the current label
9         pred = preds == label
10        target = targets == label
11        # Calculate intersection and sum for Dice score
12        intersection = (pred * target). ( )
13        dices.append((2.0 * intersection + smooth) / (pred. ( ) + target. ( ) + smooth))
14    # Return the average Dice score across all foreground labels
15    return np.mean(dices)

```

dice_multi_class(...): Calculates the Dice Similarity Coefficient (DSC), a common metric for evaluating segmentation performance. It compares the predicted segmentation (preds) with the ground truth (targets). It calculates the DSC for each distinct object label (ignoring background label 0) and returns the average score.

```

1 def show_mask(mask, ax, mask_color=None, alpha=0.5):
2     # ... (implementation for displaying a mask overlay on an image plot) ...
3     if mask_color is not None:
4         color = np.concatenate([mask_color, np.array([alpha])], axis=0)
5     else:
6         color = np.array([251/255, 252/255, 30/255, alpha]) # Default yellow
7     h, w = mask.shape[-2:]
8     mask_image = mask.reshape(h, w, 1) * color.reshape(1, 1, -1)
9     ax.imshow(mask_image)
10
11 def show_box(box, ax, edgecolor='blue'):
12     # ... (implementation for drawing a bounding box on an image plot) ...
13     x0, y0 = box[0], box[1]
14     w, h = box[2] - box[0], box[3] - box[1]
15     ax.add_patch(plt.Rectangle((x0, y0), w, h, edgecolor=edgecolor, facecolor=(0,0,0,0), lw=2))

```

show_mask(...) and **show_box(...):** These are visualization helpers designed to work with matplotlib. **show_mask** overlays a semi-transparent colored mask onto an image plot (ax), and **show_box** draws a rectangle (bounding box) onto the plot.

```

1  def resize_grayscale_to_rgb_and_resize(array, image_size):
2      """
3      Resize a 3D grayscale NumPy array to an RGB image and then resize it.
4
5      Parameters:
6      array (np.ndarray): Input array of shape (d, h, w). Grayscale slices.
7      image_size (int): Desired square size (image_size x image_size).
8
9      Returns:
10     np.ndarray: Resized array of shape (d, 3, image_size, image_size). RGB slices.
11     """
12     d, h, w = array.shape # Depth, Height, Width of the input 3D volume
13     # Initialize an empty array for the output
14     resized_array = np.zeros((d, 3, image_size, image_size))
15
16     for i in range(d): # Iterate through each slice along the depth dimension
17         # Convert the 2D grayscale slice (h, w) to a PIL Image object
18         img_pil = Image.fromarray(array[i].astype(np.uint8))
19         # Convert the PIL Image to RGB format (duplicates the single channel 3 times)
20         img_rgb = img_pil.convert("RGB")
21         # Resize the RGB image to the target square size (image_size, image_size)
22         img_resized = img_rgb.resize((image_size, image_size))
23         # Convert the resized PIL image back to a NumPy array and change layout
24         # from (image_size, image_size, 3) to (3, image_size, image_size) for PyTorch
25         img_array = np.array(img_resized).transpose(2, 0, 1)
26         # Store the processed slice in the output array
27         resized_array[i] = img_array
28
29     return resized_array

```

resize_grayscale_to_rgb_and_resize(...): This is a crucial preprocessing function. SAM2 (like the original SAM) expects RGB images of a specific square size (here, 512×512 based on the config name `--cfg configs/sam2.1_hiera_t512.yaml`). Medical images, such as CT scans, are often grayscale and have different dimensions. This function takes a 3D NumPy array (**Depth, Height, Width**) representing the grayscale volume, iterates through each 2D slice, converts it to a 3-channel RGB image (by simply duplicating the grayscale channel), resizes it to the required square `image_size`, and rearranges the dimensions to the (**Channel, Height, Width**) format expected by PyTorch models. The output is a 4D NumPy array (`depth, 3, image_size, image_size`).

```

1  def mask2D_to_bbox(gt2D, max_shift=20):
2      # ... (implementation for finding the tight bbox around a 2D mask) ...
3      # ... (adds optional random coordinate shift) ...
4      y_indices, x_indices = np.where(gt2D > 0)
5      if (x_indices) == 0: # Handle empty mask
6          return np.array([0, 0, 0, 0]) # Return zero box
7      x_min, x_max = np.amin(x_indices), np.amax(x_indices)
8      y_min, y_max = np.amin(y_indices), np.amax(y_indices)
9      H, W = gt2D.shape
10     bbox_shift = np.random.randint(0, max_shift + 1, 1)[0]
11     x_min = (0, x_min - bbox_shift)
12     x_max = (W-1, x_max + bbox_shift)
13     y_min = (0, y_min - bbox_shift)
14     y_max = (H-1, y_max + bbox_shift)
15     boxes = np.array([x_min, y_min, x_max, y_max]) # XYXY format
16     return boxes
17
18 def mask3D_to_bbox(gt3D, max_shift=20):
19     # ... (implementation for finding the tight bbox around a 3D mask) ...
20     # ... (adds optional random coordinate shift) ...
21     z_indices, y_indices, x_indices = np.where(gt3D > 0)
22     if (x_indices) == 0: # Handle empty mask
23         return np.array([0, 0, 0, 0, 0, 0]) # Return zero box
24     x_min, x_max = np.amin(x_indices), np.amax(x_indices)
25     y_min, y_max = np.amin(y_indices), np.amax(y_indices)
26     z_min, z_max = np.amin(z_indices), np.amax(z_indices)
27     D, H, W = gt3D.shape
28     bbox_shift = np.random.randint(0, max_shift + 1, 1)[0]
29     x_min = (0, x_min - bbox_shift)
30     x_max = (W-1, x_max + bbox_shift)
31     y_min = (0, y_min - bbox_shift)
32     y_max = (H-1, y_max + bbox_shift)
33     z_min = (0, z_min) # Usually no shift in Z for key slice prompting
34     z_max = (D-1, z_max)
35     boxes3d = np.array([x_min, y_min, z_min, x_max, y_max, z_max]) # XYZXYZ format
36     return boxes3d

```

mask2D_to_bbox(...) / **mask3D_to_bbox(...)**: These functions take a 2D or 3D binary mask and calculate the tightest bounding box around the foreground pixels (the object). They find the minimum and maximum x, y (and z for 3D) coordinates containing the object. Optionally, they can add a random “**shift**” to the box coordinates, slightly expanding the box. This might be used to generate bounding box prompts from existing masks, for example, for training data augmentation or simulating slightly imprecise user input. However, since this script uses `propagate_with_box=True` by default and gets the box from the CSV file, these mask-to-box functions are likely not used in the standard execution path.

Section 3: Main Processing Loop

This is the core part of the script where the actual segmentation happens. It iterates through each 3D CT scan file found in the input directory.

First, the script loads information about the lesions from the CSV. This file serves as a guide, indicating the locations of lesions within the CT scans, including details such as bounding boxes on specific slices and the optimal contrast settings (DICOM windows) for viewing them. Then, it finds all the 3D CT scan files (`.nii.gz`) in the folder provided by the user (`imgs_path`). It cleans up this list to ignore temporary files and prints the number of scans it found.

```

1  # Load metadata about lesions from a CSV file.
2  DL_info = pd.read_csv('CT_DeepLesion/DeepLesion_Dataset_Info.csv')
3
4  # Find all '.nii.gz' files in the specified image directory
5  nii_fnames = (os.listdir(imgs_path))
6  nii_fnames = [i for i in nii_fnames if i.endswith('.nii.gz')]
7  # Filter out hidden or temporary files
8  nii_fnames = [i for i in nii_fnames if not i.startswith('.')]
9  (f'Processing {len(nii_fnames)} nii files')
10
11 # Initialize an ordered dictionary to store results information
12 seg_info = OrderedDict()
13 seg_info['nii_name'] = []
14 seg_info['key_slice_index'] = []
15 seg_info['DICOM_windows'] = []
16
17 # Initialize the MedSAM2 predictor model
18 predictor = build_sam2_video_predictor_npz(model_cfg, checkpoint)

```

Finally, it sets up a dictionary (`seg_info`) to keep track of the results it generates and initializes the MedSAM2 model using the `build_sam2_video_predictor_npz` function, loading the specified configuration and pre-trained weights. This predictor is designed to work with sequences of image data, like the slices of a 3D scan.

Section 3.1: Processing Each CT Scan File

The script now enters its main loop, going through each `.nii.gz` file one by one. For each scan file, it first extracts information from the filename (like the range of slices it covers and a unique case identifier) to help find the corresponding lesion details in the previously loaded `DL_info` table.

It then loads the actual 3D image data from the `.nii.gz` file into a NumPy array using the SimpleITK library. It filters the `DL_info` table to get only the rows that match the current CT scan file. Since one scan might contain multiple lesions listed separately in the CSV, it prepares an empty 3D array (`segs_3D_volume`) of the same size as the input scan to store the combined segmentation results for all lesions found within this scan.

```

1 # Loop through each detected NIfTI file with a progress bar
2 for nii_fname in tqdm(nii_fnames):
3     # --- Start processing one NIfTI file ---
4
5     # Extract slice range and case name from the filename
6     range_suffix = re.findall(r'\d{3}-\d{3}', nii_fname)[0]
7     slice_range = range_suffix.split('-')
8     slice_range = [ (s) for s in slice_range]
9     slice_range_str = ', '.join(slice_range) # Format for matching CSV
10
11     # Load the 3D image volume and convert to NumPy array
12     nii_image = sitk.ReadImage(join(imgs_path, nii_fname))
13     nii_image_data = sitk.GetArrayFromImage(nii_image) # Shape: (Depth, Height, Width)
14
15     case_name = re.findall(r'^(\d{6}-\d{2}-\d{2})', nii_fname)[0]
16
17     # Find matching lesion entries in the CSV for this scan file
18     case_df = DL_info[
19         DL_info['File_name'].contains(case_name) &
20         DL_info['Slice_range'].contains(slice_range_str)
21     ].copy()
22
23     # Initialize an empty 3D array for the combined segmentation mask of this volume
24     segs_3D_volume = np.zeros(nii_image_data.shape, dtype=np.uint8)
25
26     # Loop through each lesion found in the CSV for this NIfTI file
27     for row_id, row in case_df.iterrows():
28         # --- Start processing one lesion within the NIfTI file ---
29         # [Lesion processing code follows inside this inner loop]
30         # ...

```

This part organizes the processing by handling one complete 3D scan at a time and preparing to process each lesion listed for that scan individually.

Section 3.2: Preprocessing for a Single Lesion

Inside the inner loop (processing one specific lesion from the CSV), the script performs preprocessing tailored to that lesion. It reads the DICOM window settings (contrast/brightness levels) from the CSV row and applies them to the entire loaded 3D volume (`nii_image_data`). This clipping and normalization step (`np.clip`, scaling to 0-255) makes the lesion easier to see according to the metadata.

It then identifies the 'key slice' index provided in the CSV and calculates its corresponding index (`key_slice_idx_offset`) within the loaded NumPy array segment. The bounding box coordinates are also extracted from the CSV and converted into the `[x_min, y_min, x_max, y_max]` (XYXY) format. The original 3D image data (after windowing/normalization) is stored in `img_3D_ori` for potential later use (like saving alongside the mask).

```

1 # 1. Load Lesion Metadata & Preprocess Image Slice for the Lesion
2     lower_bound, upper_bound = row['DICOM_windows'].split(',')
3     lower_bound, upper_bound = (lower_bound), (upper_bound)
4
5     # Apply windowing and normalize to 0-255 (uint8)
6     nii_image_data_pre = np.clip(nii_image_data, lower_bound, upper_bound)
7     nii_image_data_pre = (nii_image_data_pre - np. (nii_image_data_pre))/(np. (nii_image_data_pre)-np.
8     nii_image_data_pre = np.uint8(nii_image_data_pre)
9
10    # Get key slice index and bounding box
11    key_slice_idx_csv = (row['Key_slice_index'])
12    slice_idx_start, slice_idx_end = ( , row['Slice_range'].split(',')
13    key_slice_idx_offset = key_slice_idx_csv - slice_idx_start
14    bbox_coors = ( ( , ( , row['Bounding_boxes'].split(',') ) ) ) # ymin, xmin, ymax, xmax
15    bbox_xyxy = np.array([bbox_coors[1], bbox_coors[0], bbox_coors[3], bbox_coors[2]]) # xmin, ymin, xmax
16
17    # Store the preprocessed 3D volume data for this lesion
18    img_3D_ori = nii_image_data_pre
19    assert np. (img_3D_ori) < 256, f'Input data should be uint8 range [0, 255]'
20
21    # Get original height/width for resizing outputs later
22    key_slice_img = nii_image_data_pre[key_slice_idx_offset, :, :]
23    video_height, video_width = key_slice_img.shape[0], key_slice_img.shape[1]

```


This step focuses on preparing the image data based on the specific lesion's metadata, ensuring optimal contrast, and extracting the necessary prompt information (the bounding box).

Section 3.3: Preparing Input Tensor for MedSAM2

After basic preprocessing, the data needs further transformation to match the MedSAM2 model's expected input format. The `resize_grayscale_to_rgb_and_resize` function is called to convert the 3D grayscale volume (`img_3D_ori`) into a stack of 512×512 RGB images (even though the original was grayscale, the model expects 3 channels).

The pixel values are scaled to the range [0.0, 1.0]. This NumPy array is then converted into a PyTorch tensor and moved to the GPU (cuda). Finally, standard ImageNet normalization (subtracting the mean and dividing by the standard deviation for each channel) is applied. The result (`img_resized`) is a tensor ready to be fed into the MedSAM2 predictor.

```

1 # 2. Prepare Input for MedSAM2 Model
2     # Resize slices to 512x512 and convert to RGB format
3     img_resized = resize_grayscale_to_rgb_and_resize(img_3D_ori, 512) # Shape: (D, 3, 512, 512)
4     # Normalize pixel values to 0.0-1.0
5     img_resized = img_resized / 255.0
6     # Convert to PyTorch tensor and move to GPU
7     img_resized = torch.from_numpy(img_resized).cuda()
8
9     # Define and apply ImageNet normalization
10    img_mean=(0.485, 0.456, 0.406)
11    img_std=(0.229, 0.224, 0.225)
12    img_mean = torch.tensor(img_mean, dtype=torch.float32)[: , None, None].cuda()
13    img_std = torch.tensor(img_std, dtype=torch.float32)[: , None, None].cuda()
14    img_resized -= img_mean
15    img_resized /= img_std

```

This is the final preparation step before the actual segmentation, ensuring the data dimensions, format, and value ranges are exactly what the deep learning model expects.

Section 4: Running MedSAM2 Inference

Now, the core segmentation process begins for the current lesion. An empty 3D NumPy array (`segs_3D_lesion`) is created to store the segmentation mask specifically for this lesion. The code uses `torch.inference_mode()` and `torch.autocast` for optimized performance during inference (faster computation, potentially lower memory use with `bfloat16`).

First, the `predictor.init_state` method is called with the preprocessed image tensor (`img_resized`). This likely calculates and stores the initial image embeddings for the volume. Then, the bounding box (`bbox_xyxy`) identified earlier is provided as a prompt on the key slice (`key_slice_idx_offset`) using `predictor.add_new_points_or_box`.

The script then calls `predictor.propagate_in_video` to perform segmentation propagation forward from the key slice towards the end of the volume. This function iterates through the slices, using the model's internal memory to maintain consistency. For each slice (`out_frame_idx`), it returns the mask logits (`out_mask_logits`). These logits are thresholded (at 0.0) to create a binary mask, which is then used to mark the corresponding pixels in the `segs_3D_lesion` array.

After the forward pass, the predictor's internal state (memory) is reset using `predictor.reset_state`. The initial bounding box prompt is added again to the key slice. Then, `predictor.propagate_in_video(...)` is called to propagate segmentation backward from the key slice towards the beginning of the volume. The resulting binary masks are again used to update the `segs_3D_lesion` array. Using logical OR ensures that pixels segmented in either the forward or backward pass are included in the final lesion mask. Finally, the predictor state is reset one last time.

```

1  # 3. Run MedSAM2 Inference for the current lesion
2      segs_3D_lesion = np.zeros(nii_image_data.shape, dtype=np.uint8) # Mask for this specific lesion
3
4      # Use inference mode and mixed precision for efficiency
5      with torch.inference_mode(), torch.autocast("cuda", dtype=torch.bfloat16):
6          # Initialize predictor state with the volume data
7          inference_state = predictor.init_state(img_resized, video_height, video_width)
8
9          # Add the initial bounding box prompt on the key slice
10         if propagate_with_box:
11             predictor.add_new_points_or_box(
12                 inference_state=inference_state,
13                 frame_idx=key_slice_idx_offset,
14                 obj_id=1, # Assign ID 1 to this lesion object
15                 box=bbox_xyxy,
16             )
17         else: # gt (Alternative prompting not used by default)
18             pass
19
20         # Propagate segmentation FORWARD from the key slice
21         for out_frame_idx, _, out_mask_logits in predictor.propagate_in_video(inference_state):
22             binary_mask = (out_mask_logits[0] > 0.0).cpu().numpy()[0]
23             segs_3D_lesion[out_frame_idx, binary_mask] = 1 # Mark segmented pixels
24
25         # Reset predictor state (clear memory) before backward pass
26         predictor.reset_state(inference_state)
27
28         # Add the initial prompt AGAIN for the backward pass
29         if propagate_with_box:
30             predictor.add_new_points_or_box(
31                 inference_state=inference_state,
32                 frame_idx=key_slice_idx_offset,
33                 obj_id=1,
34                 box=bbox_xyxy,
35             )
36         else: # gt
37             pass
38
39         # Propagate segmentation BACKWARD from the key slice
40         for out_frame_idx, _, out_mask_logits in predictor.propagate_in_video(inference_state, reverse=True):
41             binary_mask = (out_mask_logits[0] > 0.0).cpu().numpy()[0]
42             segs_3D_lesion[out_frame_idx, binary_mask] = 1 # Update mask (union)
43
44         # Reset state after finishing this lesion
45         predictor.reset_state(inference_state)

```

This bidirectional propagation is the key step where MedSAM2 uses the initial prompt and its temporal consistency mechanisms to segment the lesion across multiple slices.

Section 4.1: Post-processing and Combining Results

After the forward and backward propagation for a single lesion, the resulting mask (`segs_3D_lesion`) might contain small, disconnected regions or noise. The script checks if any segmentation actually happened (`np.max(segs_3D_lesion) > 0`). If so, it calls the `getLargestCC` function to perform post-processing, keeping only the largest connected component of the segmented pixels.

This cleans up the result. Then, this cleaned-up mask for the current lesion is combined with the main segmentation mask for the entire **NIfTI** volume (`segs_3D_volume`) using a logical **OR** operation. This ensures that if multiple lesions are processed for the same scan file, their segmentations are accumulated into the final output mask.

```

1  # 4. Post-process the segmentation for the current lesion
2      if np. (segs_3D_lesion) > 0:
3          # Keep only the largest connected component (removes noise)
4          segs_3D_lesion = getLargestCC(segs_3D_lesion)
5          segs_3D_lesion = np.uint8(segs_3D_lesion)
6
7      # 5. Combine the current lesion's segmentation with the volume's segmentation
8      # Use logical OR to merge masks if multiple lesions exist in the volume
9      segs_3D_volume = np.logical_or(segs_3D_volume, segs_3D_lesion).astype(np.uint8)
10
11  # --- End processing one lesion --- (Inner loop finishes here) ---

```

This step refines the individual lesion segmentation and merges it into the final output for the entire scan volume.

Section 4.2: Saving Final Results

Once the inner loop finishes (meaning all lesions listed for the current NIfTI file in the CSV have been processed and their masks combined into `segs_3D_volume`), the script saves the results. It converts the final NumPy segmentation array (`segs_3D_volume`) back into a SimpleITK image object. Critically, it copies the spatial metadata (like spacing, origin, and orientation) from the original input NIfTI image (`nii_image`) to the new mask image (`sitk_mask`). This ensures the saved mask aligns correctly with the original scan in medical viewers.

It also saves the preprocessed (windowed) version of the input image (`img_3D_ori` from the last lesion processed) for reference. Filenames are constructed based on the original input filename, adding suffixes like `_img.nii.gz` and `_mask.nii.gz` (incorporating the key slice index from the last lesion into the mask filename). Finally, it records information about the saved mask file (filename, key slice index, DICOM window used for the last lesion) into the `seg_info` dictionary.

```

1  # --- End processing one NIfTI file --- (Outer loop continues after this) ---
2
3      # 6. Save Results for the entire NIfTI volume
4      # Convert final NumPy mask to SimpleITK image
5      sitk_mask = sitk.GetImageFromArray(segs_3D_volume)
6      # Copy spatial metadata from original NIfTI
7      sitk_mask.CopyInformation(nii_image)
8
9      # Prepare the preprocessed image (from last lesion) for saving
10     sitk_image_preprocessed = sitk.GetImageFromArray(img_3D_ori)
11     sitk_image_preprocessed.CopyInformation(nii_image)
12
13     # Define output filenames
14     key_slice_idx_csv = (row['Key_slice_index']) # Key slice from the last processed row
15     save_seg_name = nii_fname.split('.')[0] + f'_k{key_slice_idx_csv}_mask.nii.gz'
16     save_img_name = nii_fname.replace('.nii.gz', '_img.nii.gz')
17
18     # Write the preprocessed image and the final segmentation mask to disk
19     sitk.WriteImage(sitk_image_preprocessed, os.path.join(pred_save_dir, save_img_name))
20     sitk.WriteImage(sitk_mask, os.path.join(pred_save_dir, save_seg_name))
21
22     # Record metadata about the saved segmentation
23     seg_info['nii_name'].append(save_seg_name)
24     seg_info['key_slice_index'].append(key_slice_idx_csv)
25     seg_info['DICOM_windows'].append(row['DICOM_windows'])
26
27  # --- (Outer loop continues to the next nii_fname) ---

```

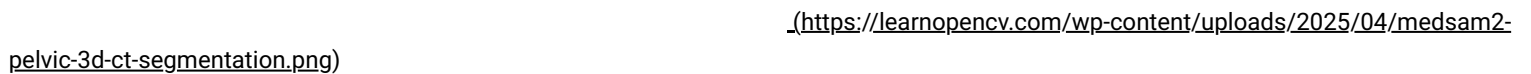
This is where the final output for each processed CT scan volume, along with relevant metadata, is generated and stored.

Section 4.3: Saving Summary Information

After the main outer loop has finished processing all the NIfTI files in the input directory, the script takes the `seg_info` dictionary, which has collected the filename, key slice index, and DICOM window for each generated segmentation mask, and converts it into a Pandas DataFrame. This DataFrame is then saved as a CSV file (named `tiny_seg_info202412.csv` in the specified output directory). This summary file provides a convenient way to track which output mask corresponds to which input file and key slice prompt.

```
1 # After processing all NIfTI files, save the collected segmentation info to a CSV
2 seg_info_df = pd.DataFrame(seg_info)
3 seg_info_df.to_csv(join(pred_save_dir, 'tiny_seg_info202412.csv'), index=False)
```

And here is the result:

 (https://learnopencv.com/wp-content/uploads/2025/04/medsam2-pelvic-3d-ct-segmentation.png)

3D CT Segmentation of Pelvic Lesion

You can see the model has segmented a small structure in the pelvis, most likely the **bladder** or possibly a **prostate tumor** or **pelvic mass**, depending on the clinical task. Isn't it cool! This can save a lot of time for doctors and surgeons, helping them make decisions quickly, which can save millions of lives and make our healthcare system smarter and more efficient.

Video Segmentation – Heart Echo

In this section, we will use an echocardiogram (or “echo”) video of a patient to examine the heart, like the left ventricle (the main pumping chamber), its muscle wall, and nearby areas.

An **echocardiogram** (A *specialized type of ultrasound* that focuses specifically on imaging the **heart**) image, specifically, it appears to be a **4-chamber (4CH) view** of the heart, often used in cardiac ultrasound (echocardiography).

In a 4CH view, you typically visualize:

- The **left and right atria** (top chambers)
- The **left and right ventricles** (bottom chambers)
- The **interatrial** and **interventricular septa** (walls separating chambers)

We will try with this 4CH video and see how MedSAM2 segments the heart components.

Section 1: Imports and Setup

This part imports the necessary libraries, including `argparse`, `os`, `collections`, `defaultdict`, `NumPy`, and `PyTorch`. It specifically imports `build_sam2_video_predictor` from the MedSAM2 library, which is the main class responsible for handling video segmentation inference.

```

1 # Copyright (c) Meta Platforms, Inc. and affiliates.
2 # All rights reserved.
3
4 # This source code is licensed under the license found in the
5 # LICENSE file in the root directory of this source tree.
6
7 import argparse
8 import os
9 from collections import defaultdict
10
11 import numpy as np
12 import torch
13 from PIL import Image
14 from sam2.build_sam import build_sam2_video_predictor
15
16 # the PNG palette for DAVIS 2017 dataset
17 DAVIS_PALETTE = b"\x00\x00\...\xe0\xe0\xe0@\` \xc0\xe0\` \xc0\` \xe0\xc0\xe0\xe0\xc0"

```

The script also defines `DAVIS_PALETTE`, which is a specific color map (palette) used in the DAVIS 2017 video segmentation benchmark dataset. This palette allows saving multi-object segmentation masks as a single PNG image where each object ID corresponds to a unique color. This is useful for visualization and evaluation on standard benchmarks.

Section 2: Helper Functions for PNG Mask Handling

This section defines several functions specifically for reading and writing segmentation masks stored as PNG image files. These formats are common in video object segmentation datasets.

```

1 def load_ann_png(path):
2     """Load a PNG file as a mask and its palette."""
3     # Open the PNG image file using PIL
4     mask = Image.open(path)
5     # Get the color palette embedded in the PNG (if it exists)
6     palette = mask.getpalette()
7     # Convert the PIL image to a NumPy array of unsigned 8-bit integers (uint8)
8     mask = np.array(mask).astype(np.uint8)
9     # Return both the mask data (as a NumPy array) and the palette
10    return mask, palette

```

load_ann_png(...): This function reads a PNG file that's expected to be a segmentation mask, often using a specific color palette (like the `DAVIS_PALETTE` defined earlier). It uses the Pillow (PIL) library to open the image, extracts the mask data as a NumPy array (where pixel values typically represent object IDs), and also extracts the color palette associated with the image.

```

1 def save_ann_png(path, mask, palette):
2     """Save a mask as a PNG file with the given palette."""
3     # Ensure the mask data is uint8 and 2-dimensional (Height x Width)
4     assert mask.dtype == np.uint8
5     assert mask.ndim == 2
6     # Convert the NumPy array back to a PIL Image object
7     output_mask = Image.fromarray(mask)
8     # Apply the provided color palette to the image
9     output_mask.putpalette(palette)
10    # Save the image to the specified path
11    output_mask.save(path)

```

save_ann_png(...): This is the counterpart to `load_ann_png`. It takes a NumPy array representing a mask and a color palette, creates a PIL Image object from the mask data, applies the palette to it, and saves it as a PNG file at the given path.

```

1 def get_per_obj_mask(mask):
2     """Split a mask into per-object masks."""
3     # Find all unique pixel values (object IDs) in the mask
4     object_ids = np.unique(mask)
5     # Filter out the background ID (usually 0)
6     object_ids = object_ids[object_ids > 0].tolist()
7     # Create a dictionary: key is object ID, value is a binary (True/False) mask
8     # where True indicates pixels belonging to that specific object ID.
9     per_obj_mask = {object_id: (mask == object_id) for object_id in object_ids}
10    return per_obj_mask

```

get_per_obj_mask(...): This function takes a mask (like one loaded by `load_ann_png`) where different objects are represented by different pixel values (e.g., object 1 is value 1, object 2 is value 2). It identifies all unique non-zero object IDs present and creates a dictionary. Each key in the dictionary is an object ID, and the corresponding value is a binary (True/False) mask indicating only the pixels belonging to that specific object.

```

1 def put_per_obj_mask(per_obj_mask, height, width):
2     """Combine per-object masks into a single mask."""
3     # Create an empty mask filled with zeros (background)
4     mask = np.zeros((height, width), dtype=np.uint8)
5     # Get object IDs from the input dictionary, sort them in reverse order
6     # (so higher IDs overwrite lower IDs if masks overlap, common in VOS)
7     object_ids = sorted(per_obj_mask.keys(), reverse=True)
8     for object_id in object_ids:
9         # Get the binary mask for the current object ID
10        object_mask = per_obj_mask[object_id]
11        # Ensure it has the correct shape (Height x Width)
12        object_mask = object_mask.reshape(height, width)
13        # Assign the object ID value to the pixels where the binary mask is True
14        mask[object_mask] = object_id
15    return mask

```

put_per_obj_mask(...): This does the reverse. It takes a dictionary of binary masks (like the one produced by `get_per_obj_mask` or the output from the model) and combines them into a single mask image. Pixels belonging to object ID *n* will be assigned the value *n* in the output mask. It handles potential overlaps by processing objects in reverse sorted order of IDs, meaning higher IDs take precedence.

```

1 def load_masks_from_dir(
2     input_mask_dir, video_name, frame_name, per_obj_png_file, allow_missing=False
3 ):
4     """Load masks from a directory as a dict of per-object masks."""
5     # Case 1: Masks are combined in a single palettized PNG per frame (like DAVIS)
6     if not per_obj_png_file:
7         input_mask_path = os.path.join(input_mask_dir, video_name, f"{frame_name}.png")
8         # Optionally skip if the mask file doesn't exist
9         if allow_missing and not os.path.exists(input_mask_path):
10            return {}, None
11        # Load the combined mask and its palette
12        input_mask, input_palette = load_ann_png(input_mask_path)
13        # Split into individual binary masks per object
14        per_obj_input_mask = get_per_obj_mask(input_mask)
15    # Case 2: Each object has its own PNG file in a subdirectory (like SA-V)
16    else:
17        per_obj_input_mask = {}
18        input_palette = None # Palette is not usually stored per-object
19        # Iterate through object subdirectories (named like '001', '002', ...)
20        for object_name in os.listdir(os.path.join(input_mask_dir, video_name)):
21            object_id = int(object_name)
22            input_mask_path = os.path.join(
23                input_mask_dir, video_name, object_name, f"{frame_name}.png"
24            )
25            # Optionally skip if the mask file doesn't exist
26            if allow_missing and not os.path.exists(input_mask_path):
27                continue
28            # Load the individual object mask (palette might be loaded but usually ignored)
29            input_mask, input_palette = load_ann_png(input_mask_path)
30            # Store the binary mask (pixels > 0 are foreground)
31            per_obj_input_mask[object_id] = input_mask > 0
32
33    return per_obj_input_mask, input_palette

```

load_masks_from_dir(...): This is a higher-level function that uses the previous helpers. It reads mask information for a specific `frame_name` within a `video_name` directory. It adapts its behavior based on the `per_obj_png_file` flag:

- If **False** (default), it expects a single PNG file (like `frame_name.png`) containing all object masks distinguished by pixel values (like DAVIS format). It uses `load_ann_png` and `get_per_obj_mask`.
- If **True**, it expects subdirectories for each object (e.g., `001/`, `002/`) inside the video directory, each containing individual PNG mask files (`frame_name.png`). It loads these individual files. It returns a dictionary of binary masks per object ID and potentially the palette from the input.

```

1  def save_palette_masks_to_dir(
2      output_mask_dir,
3      video_name,
4      frame_name,
5      per_obj_output_mask,
6      height,
7      width,
8      per_obj_png_file,
9      output_palette,
10 ):
11     """Save masks to a directory as PNG files with palette."""
12     # Create the output directory for the video if it doesn't exist
13     os.makedirs(os.path.join(output_mask_dir, video_name), exist_ok=True)
14     # Case 1: Save as a single combined palettized PNG
15     if not per_obj_png_file:
16         # Combine binary masks into a single mask with object IDs as pixel values
17         output_mask = put_per_obj_mask(per_obj_output_mask, height, width)
18         output_mask_path = os.path.join(
19             output_mask_dir, video_name, f"{frame_name}.png"
20         )
21         # Save the combined mask using the specified palette
22         save_ann_png(output_mask_path, output_mask, output_palette)
23     # Case 2: Save each object's mask as a separate PNG
24     else:
25         for object_id, object_mask in per_obj_output_mask.items():
26             object_name = f"{object_id:03d}" # Format object ID (e.g., 1 -> '001')
27             # Create subdirectory for the object if needed
28             os.makedirs(
29                 os.path.join(output_mask_dir, video_name, object_name),
30                 exist_ok=True,
31             )
32             # Reshape the binary mask and convert to uint8
33             output_mask = object_mask.reshape(height, width).astype(np.uint8)
34             output_mask_path = os.path.join(
35                 output_mask_dir, video_name, object_name, f"{frame_name}.png"
36             )
37             # Save the individual object mask with the palette
38             # (Note: palette might not be meaningful here, but function requires it)
39             save_ann_png(output_mask_path, output_mask, output_palette)

```

save_palette_masks_to_dir(...): Saves the predicted masks (`per_obj_output_mask`, which is a dictionary of binary masks) as PNG files with a color palette (either the one loaded from the input or the default `DAVIS_PALETTE`). It also handles the two storage formats based on `per_obj_png_file`: either combining masks into one palettized PNG using `put_per_obj_mask`, and `save_ann_png`, or saving each object's binary mask into its respective subdirectory as a separate palettized PNG.


```

1  def save_masks_to_dir(
2      output_mask_dir,
3      video_name,
4      frame_name,
5      per_obj_output_mask,
6      height,
7      width,
8      per_obj_png_file,
9  ):
10     """Save masks to a directory as greyscale PNG files (no palette)."""
11     # Create the output directory for the video if it doesn't exist
12     os.makedirs(os.path.join(output_mask_dir, video_name), exist_ok=True)
13     # Case 1: Save as a single combined grayscale PNG
14     if not per_obj_png_file:
15         # Combine binary masks into a single mask with object IDs as pixel values
16         output_mask = put_per_obj_mask(per_obj_output_mask, height, width)
17         output_mask_path = os.path.join(
18             output_mask_dir, video_name, f"{frame_name}.png"
19         )
20         # Ensure correct data type and dimensions
21         assert output_mask.dtype == np.uint8
22         assert output_mask.ndim == 2
23         # Convert to PIL Image and save (will be grayscale based on object IDs)
24         output_mask = Image.fromarray(output_mask)
25         output_mask.save(output_mask_path)
26     # Case 2: Save each object's mask as a separate grayscale PNG
27     else:
28         for object_id, object_mask in per_obj_output_mask.items():
29             object_name = f"{object_id:03d}" # Format object ID
30             # Create subdirectory for the object if needed
31             os.makedirs(
32                 os.path.join(output_mask_dir, video_name, object_name),
33                 exist_ok=True,
34             )
35             # Reshape binary mask, convert to uint8 (True becomes 1, False becomes 0)
36             output_mask = object_mask.reshape(height, width).astype(np.uint8)
37             output_mask_path = os.path.join(
38                 output_mask_dir, video_name, object_name, f"{frame_name}.png"
39             )
40             # Ensure correct data type and dimensions
41             assert output_mask.dtype == np.uint8
42             assert output_mask.ndim == 2
43             # Convert to PIL Image and save (will be grayscale, mostly 0s and 1s)
44             output_mask = Image.fromarray(output_mask)
45             output_mask.save(output_mask_path)

```

save_masks_to_dir(...): This function is similar to `save_palette_masks_to_dir`, but it saves the output masks as grayscale PNGs without applying a color palette. In the combined format, the pixel value will be the object ID. In the per-object format, the pixel value will typically be 1 for the object and 0 for the background. This might be used for storing raw predictions before visualization or evaluation that requires a specific palette format.

These functions provide the necessary tools to handle the input mask prompts and save the output segmentations in standard formats commonly used for video object segmentation tasks.

Section 3: VOS Inference

Section 3.1: VOS Inference (Standard Case)

This section defines the `vos_inference` function, which handles the standard video object segmentation task. It assumes that all objects you want to track are present and visible in the initial frame(s) for which you provide masks. The decorators `@torch.inference_mode(...)` and `@torch.autocast(...)` tell PyTorch to optimize for inference (e.g., by not tracking gradients) and to automatically use lower-precision calculations (like `bfloat16`) if possible, making it run faster and use less memory on compatible GPUs.

```

1  @torch.inference_mode()
2  @torch.autocast(device_type="cuda", dtype=torch.bfloat16)
3  def vos_inference(
4      predictor,                # The initialized MedSAM2 video predictor model
5      base_video_dir,          # Directory containing folders of video frames
6      input_mask_dir,          # Directory containing folders of input masks (PNGs)
7      output_mask_dir,         # Directory where output masks will be saved
8      video_name,              # Name of the specific video folder to process
9      score_thresh=0.0,        # Threshold for converting logits to binary masks (default 0.0)
10     use_all_masks=False,      # If True, use masks from all frames in input_mask_dir as prompts
11                               # If False (default), only use the mask from the very first frame
12     per_obj_png_file=False,    # If True, input/output masks are one PNG per object per frame
13                               # If False (default), masks are one combined PNG per frame
14     save_palette_png=False,    # If True, save output masks with a color palette
15                               # If False (default), save as grayscale masks
16 ):
17     """Run inference on a single video with the given predictor."""
18     # Construct the full path to the video frames directory
19     video_dir = os.path.join(base_video_dir, video_name)
20     # Get a sorted list of frame filenames (without extension)
21     frame_names = [
22         os.path.splitext(p)[0]
23         for p in os.listdir(video_dir)
24         if os.path.splitext(p)[-1] in [".jpg", ".jpeg", ".JPG", ".JPEG"]
25     ]
26     frame_names = sorted(frame_names)
27
28     # Initialize the predictor's internal state for this video. This involves
29     # loading the video frames (potentially asynchronously) and preparing the model.
30     inference_state = predictor.init_state(
31         video_path=video_dir, async_loading_frames=False # False means load all frames now
32     )
33     # Store the video dimensions for later use when saving masks
34     height = inference_state["video_height"]
35     width = inference_state["video_width"]
36     input_palette = None # Will store the palette if loaded from input mask

```

This initial part sets up the function, finds all the frame image files for the specified video, and initializes the MedSAM2 predictor's state, which includes loading the video frames and getting their dimensions.

Next, the function determines which frame(s) contain the initial mask prompts that will guide the segmentation. By default (`use_all_masks=False`), it assumes only the very first frame (`frame_names[0]`) has an input mask. If `use_all_masks=True`, it searches the `input_mask_dir` for all available mask files for this video, and uses all of them as initial prompts. The indices of these prompt frames are stored in `input_frame_inds`.

```

1  # fetch mask inputs from input_mask_dir (either only mask for the first frame, or all available masks)
2  if not use_all_masks:
3      # Default: use only the first frame's mask as the input prompt
4      input_frame_inds = [0]
5  else:
6      # Option: use all available mask files as input prompts
7      if not per_obj_png_file: # Case 1: Combined mask PNG per frame
8          input_frame_inds = [
9              idx
10             for idx, name in (frame_names)
11             # Check if a mask file exists for this frame index
12             if os.path.exists(
13                 os.path.join(input_mask_dir, video_name, f"{name}.png")
14             )
15         ]
16      else: # Case 2: Separate mask PNG per object per frame
17          input_frame_inds = [
18              idx
19              # Iterate through object subdirectories
20              for object_name in os.listdir(os.path.join(input_mask_dir, video_name))
21              # Iterate through frame indices and names
22              for idx, name in (frame_names)
23              # Check if a mask file exists for this object and frame index
24              if os.path.exists(
25                  os.path.join(input_mask_dir, video_name, object_name, f"{name}.png")
26              )
27          ]
28      # Ensure at least one input mask was found
29      if (input_frame_inds) == 0:
30          raise RuntimeError(
31              f"In {video_name=}, got no input masks in {input_mask_dir=}. "
32              "Please make sure the input masks are available in the correct format."
33          )
34      # Get unique frame indices and sort them
35      input_frame_inds = sorted(input_frame_inds)

```

This logic allows flexibility in how the initial segmentation guidance is provided – either just the first frame or multiple frames spread throughout the video.

Now, the script iterates through the identified `input_frame_inds`. For each prompt frame, it uses the `load_masks_from_dir` helper function (explained in the previous section) to load the mask(s). It gets back a dictionary `per_obj_input_mask` where keys are object IDs and values are binary masks. It also checks if any new object IDs appear in later prompt frames that weren't in the very first one – this standard `vos_inference` function assumes all objects are present initially and raises an error if this isn't true (suggesting the user should use the other inference function or a flag). For each object mask loaded, it calls `predictor.add_new_mask` to feed this initial guidance into the MedSAM2 model's state for that specific object ID and frame index.

```

1 # add those input masks to SAM 2 inference state before propagation
2 object_ids_set = None # To keep track of all object IDs seen so far
3 for input_frame_idx in input_frame_inds:
4     try:
5         # Load mask(s) for the current prompt frame
6         per_obj_input_mask, input_palette = load_masks_from_dir(
7             input_mask_dir=input_mask_dir,
8             video_name=video_name,
9             frame_name=frame_names[input_frame_idx],
10            per_obj_png_file=per_obj_png_file,
11        )
12    except FileNotFoundError as e:
13        # Handle error if a mask file is expected but not found
14        raise RuntimeError(
15            f"In {video_name=}, failed to load input mask for frame {input_frame_idx=}. "
16            "Please add the '--track_object_appearing_later_in_video' flag "
17            # ... (rest of error message) ...
18        ) from e
19
20 # Get the set of object IDs from the first prompt frame encountered
21 if object_ids_set is None:
22     object_ids_set = (per_obj_input_mask)
23
24 # Add each loaded object mask to the predictor's state
25 for object_id, object_mask in per_obj_input_mask.items():
26     # Check if a new object ID appears only in a later frame (not allowed here)
27     if object_id not in object_ids_set:
28         raise RuntimeError(
29             f"In {video_name=}, got a new {object_id=} appearing only in a "
30             f"later {input_frame_idx=} (but not appearing in the first frame). "
31             # ... (rest of error message) ...
32         )
33     # Add the mask to the predictor state
34     predictor.add_new_mask(
35         inference_state=inference_state,
36         frame_idx=input_frame_idx,
37         obj_id=object_id,
38         mask=object_mask, # This is the binary mask (True/False)
39     )
40
41 # Check if any objects were actually loaded
42 if object_ids_set is None or (object_ids_set) == 0:
43     raise RuntimeError(
44         f"In {video_name=}, got no object ids on {input_frame_inds=}. "
45         # ... (rest of error message) ...
46     )

```

After loading all initial prompts, the script is ready to track the objects through the rest of the video.

The core tracking happens in the `predictor.propagate_in_video` loop. This function internally iterates through all frames of the video (starting from the earliest prompt frame). For each frame (`out_frame_idx`), it uses the MedSAM2 model (with its memory mechanism) to predict the masks for all the objects it's currently tracking (`out_obj_ids`). It returns the raw prediction scores (logits) for each object (`out_mask_logits`). The script then converts these logits into binary masks by comparing them against the `score_thresh` (defaulting to 0.0). These binary masks are stored in the `video_segments` dictionary, keyed by the frame index.

```

1 # run propagation throughout the video and collect the results in a dict
2 os.makedirs(os.path.join(output_mask_dir, video_name), exist_ok=True)
3 # Use the palette from input if available, otherwise default to DAVIS palette
4 output_palette = input_palette or DAVIS_PALETTE
5 video_segments = {} # Stores per-frame segmentation results {frame_idx: {obj_id: mask}}
6
7 # The main loop where the model propagates masks frame by frame
8 for out_frame_idx, out_obj_ids, out_mask_logits in predictor.propagate_in_video(
9     inference_state
10 ):
11     # Convert predicted logits to binary masks for each object
12     per_obj_output_mask = {
13         out_obj_id: (out_mask_logits[i] > score_thresh).cpu().numpy()
14         for i, out_obj_id in enumerate(out_obj_ids)
15     }
16     # Store the binary masks for the current frame
17     video_segments[out_frame_idx] = per_obj_output_mask

```

This loop performs the actual segmentation across the video sequence using the model's temporal consistency features.

Finally, the script iterates through the collected `video_segments`. For each frame, it takes the dictionary of per-object binary masks and saves them to the `output_mask_dir`. Depending on the `save_palette_png` flag, it either calls `save_palette_masks_to_dir` (to save potentially colorful, combined, or separate PNGs using a palette) or `save_masks_to_dir` (to save grayscale PNGs without a palette). The `per_obj_png_file` flag again determines whether the output is a single combined mask file per frame or separate files per object per frame.

```

1  # write the output masks as palette PNG files to output_mask_dir
2  for out_frame_idx, per_obj_output_mask in video_segments.items():
3      if save_palette_png:
4          # Option 1: save palette PNG prediction results
5          save_palette_masks_to_dir(
6              output_mask_dir=output_mask_dir,
7              video_name=video_name,
8              frame_name=frame_names[out_frame_idx], # Use original frame name
9              per_obj_output_mask=per_obj_output_mask, # Dict of {obj_id: binary_mask}
10             height=height,
11             width=width,
12             per_obj_png_file=per_obj_png_file, # Save format flag
13             output_palette=output_palette,      # Palette to use
14         )
15     else:
16         # Option 2: save raw grayscale prediction results (no palette)
17         save_masks_to_dir(
18             output_mask_dir=output_mask_dir,
19             video_name=video_name,
20             frame_name=frame_names[out_frame_idx],
21             per_obj_output_mask=per_obj_output_mask,
22             height=height,
23             width=width,
24             per_obj_png_file=per_obj_png_file,
25         )

```

This function provides the standard workflow for VOS, where the initial frame contains all necessary object information.

Section 3.2: VOS Inference (Objects Appearing Later)

This section defines `vos_separate_inference_per_object`, designed for VOS datasets where new objects might appear later in the video (not just in the first frame), like LVOS or YouTube-VOS. The key difference is that it runs the tracking process independently for each object, starting from the frame where that object first appears.

```

1  # --- Collects all masks for all objects across all frames first ---
2  inputs_per_object = defaultdict( )
3  for idx, name in (frame_names):
4      # ... (loads masks for frame 'idx') ...
5      for object_id, object_mask in per_obj_input_mask.items():
6          # ... (stores mask in inputs_per_object[object_id][idx]) ...
7
8  # --- Runs inference separately for each object ---
9  object_ids = (inputs_per_object)
10 output_scores_per_object = defaultdict( ) # Store raw scores per object
11 for object_id in object_ids:
12     input_frame_inds = (inputs_per_object[object_id]) # Frames for this object
13     predictor.reset_state(inference_state) # <<< Reset state for each object
14     for input_frame_idx in input_frame_inds:
15         # Add only this object's mask(s)
16         predictor.add_new_mask(
17             # ...
18         )
19
20     # Propagate starting from this object's first appearance
21     for out_frame_idx, _, out_mask_logits in predictor.propagate_in_video(
22         inference_state,
23         start_frame_idx= (input_frame_inds), # <<< Start from object's first frame
24         # ...
25     ):
26         # Store raw scores (logits) for this object
27         obj_scores = out_mask_logits.cpu().numpy()
28         output_scores_per_object[object_id][out_frame_idx] = obj_scores
29
30 # --- Post-processing: consolidate scores and apply constraints ---
31 video_segments = {}
32 for frame_idx in (frame_names):
33     # ... (gather scores for all objects for this frame_idx) ...
34
35     if not per_obj_png_file:
36         # Resolve overlaps using predictor's internal logic
37         scores = predictor._apply_non_overlapping_constraints(scores) # <<< Constraint step
38
39     # Threshold final, potentially constrained, scores
40     per_obj_output_mask = {
41         object_id: (scores[i] > score_thresh).cpu().numpy()
42         # ...
43     }
44     video_segments[frame_idx] = per_obj_output_mask

```

The primary upgrade in `vos_separate_inference_per_object` is – its ability to handle videos where objects appear at different times, not just in the first frame. Unlike the standard `vos_inference`, it first scans the entire video duration to collect all initial mask prompts provided for each object across all frames. Then, it processes each object independently: for every object, it resets the model's memory, adds only that specific object's initial mask(s), and starts the tracking propagation specifically from the frame where that object first appeared.

The raw prediction scores for each object are stored separately. Finally, after tracking all objects individually, it combines these scores frame-by-frame, resolves potential overlaps between different object predictions using `_apply_non_overlapping_constraints`, and then generates the final segmentation masks. This per-object approach with targeted start times and later consolidation ensures accurate tracking even when objects enter the scene mid-video. The rest of the workflow remains the same with the standard `vos_inference`.

Section 4: Main Execution Block

This final part of the script defines the main function that orchestrates the whole process and parses the command-line arguments provided by the user when the script is run.

Section 4.1: Argument Parsing (main function)

The `main()` function starts by setting up an `ArgumentParser`. This is how the script understands instructions given to it through the command line. It defines various arguments that control the script's behavior. After defining these arguments, `parser.parse_args()` reads the values provided by the user on the command line (or uses the defaults if not provided) and stores them in the `args` object.

```

1  def main():
2      parser = argparse.ArgumentParser()
3      # --- Define all command-line arguments ---
4      parser.add_argument(
5          "--sam2_cfg",
6          default="configs/sam2.1_hiera_t512.yaml",
7          help="MedSAM2 model configuration file",
8      )
9      # more arguments here...
10
11     parser.add_argument(
12         "--use_vos_optimized_video_predictor", action="store_true",
13         help="whether to use vos optimized video predictor with all modules compiled",
14     )
15     # --- Parse the arguments ---
16     args = parser.parse_args()
17
18     # --- (Code continues below) ---

```

This sets up the configuration for the rest of the main function based on user input. We are skipping the explanation for these arguments, we are moving forward to the main execution part.

Section 4.2: Model Initialization and Video List Preparation

Inside `main()`, after parsing arguments, the script initializes the MedSAM2 video predictor. It uses `build_sam2_video_predictor`, passing the model config path (`args.sam2_cfg`), checkpoint path (`args.sam2_checkpoint`), and other relevant flags (`apply_postprocessing`, `use_vos_optimized_video_predictor`). It also constructs `hydra_overrides_extra` based on the `per_obj_png_file` argument to configure the model's handling of potentially overlapping masks internally.

Next, it determines the list of videos (`video_names`) to process. If `args.video_list_file` was provided, it reads the video names from that text file. Otherwise, it scans the `args.base_video_dir` and lists all subdirectories, assuming each subdirectory corresponds to a video. It then prints messages indicating whether it's using only the first frame mask or all masks as input, and lists the videos it's about to process.

```

1  # --- (Code continues from above) ---
2
3  # Configure model overrides based on arguments
4  hydra_overrides_extra = [
5      "+model.non_overlap_masks=" + ("false" if args.per_obj_png_file else "true")
6  ]
7  # Initialize the MedSAM2 video predictor model
8  predictor = build_sam2_video_predictor(
9      config_file=args.sam2_cfg,
10     ckpt_path=args.sam2_checkpoint,
11     apply_postprocessing=args.apply_postprocessing,
12     hydra_overrides_extra=hydra_overrides_extra,
13     vos_optimized=args.use_vos_optimized_video_predictor,
14 )
15
16 # Print status message about input mask usage
17 if args.use_all_masks:
18     ("using all available masks in input_mask_dir as input to the MedSAM2 model")
19 else:
20     ("using only the first frame's mask in input_mask_dir as input to the MedSAM2 model")
21
22 # Determine the list of video names to process
23 if args.video_list_file is not None:
24     # Read video names from the specified file
25     with (args.video_list_file, "r") as f:
26         video_names = [v.strip() for v in f.readlines()]
27 else:
28     # Get video names by listing subdirectories in the base video directory
29     video_names = [
30         p
31         for p in os.listdir(args.base_video_dir)
32         if os.path.isdir(os.path.join(args.base_video_dir, p))
33     ]
34     ("running inference on {len(video_names)} videos:\n{video_names}")
35
36 # --- (Code continues below) ---

```

This part loads the actual model and figures out which videos need to be processed.

Section 5: Running Inference on Videos

This is the main processing loop within the main function. It iterates through each `video_name` in the prepared list. For each video, it prints a status message. Then, it checks the `args.track_object_appearing_later_in_video` flag.

- If the flag is `False` (default), it calls the `vos_inference` function (explained in Section 3.1), passing the predictor and all the relevant arguments (directories, video name, flags).
- If the flag is `True`, it calls the `vos_separate_inference_per_object` function (explained in Section 3.2) instead, again passing the necessary arguments.

After the loop finishes processing all videos, it prints a final confirmation message indicating where the output masks were saved.


```

1  # --- (Code continues from above) ---
2
3  # Loop through each video name
4  for n_video, video_name in enumerate(video_names):
5      (f"\n{n_video + 1}/{len(video_names)} - running on {video_name}")
6
7      # Choose the appropriate inference function based on the flag
8      if not args.track_object_appearing_later_in_video:
9          # Standard VOS: assume all objects appear in the first frame prompt
10         vos_inference(
11             predictor=predictor,
12             base_video_dir=args.base_video_dir,
13             input_mask_dir=args.input_mask_dir,
14             output_mask_dir=args.output_mask_dir,
15             video_name=video_name,
16             score_thresh=args.score_thresh,
17             use_all_masks=args.use_all_masks,
18             per_obj_png_file=args.per_obj_png_file,
19             save_palette_png=args.save_palette_png,
20         )
21     else:
22         # VOS variant: handle objects appearing later by processing each object separately
23         vos_separate_inference_per_object(
24             predictor=predictor,
25             base_video_dir=args.base_video_dir,
26             input_mask_dir=args.input_mask_dir,
27             output_mask_dir=args.output_mask_dir,
28             video_name=video_name,
29             score_thresh=args.score_thresh,
30             use_all_masks=args.use_all_masks,
31             per_obj_png_file=args.per_obj_png_file,
32             # Note: vos_separate_inference_per_object implicitly saves palette PNGs
33         )
34
35     # Final confirmation message
36     (
37         f"completed inference on {len(video_names)} videos -- "
38         f"output masks saved to {args.output_mask_dir}"
39     )
40
41 if __name__ == "__main__":
42     main()

```

For visualization, we overlaid those masks with the input images and took three frames from different timestamps. Here is the result:

(<https://learnopencv.com/wp-content/uploads/2025/04/medsam2-heart-echo-ultrasound-segmentation.png>),

Cardiac Ultrasound (Echo) Segmentation

As you can see, the model can segment the heart components correctly:

- **Red Region:** Likely the **left ventricle cavity**, where blood fills during diastole and is pumped out during systole.
- **Green Region:** Appears to be the **myocardium** – the muscular wall of the left ventricle.
- **Yellow Region:** Probably the **left ventricular outflow tract (LVOT)** or part of the **mitral valve apparatus**, depending on the specific anatomical view.

So long, right! We have provided the notebook versions of these scripts, which you can run easily on the flow.

PS: You can download all the notebooks in the [Download Code](#) button; these are slightly modified and adapted from the original MedSAM2 repository.

Now let's summarize what we have learned so far!

TL;DR

- **The Challenge in Medical Imaging** – Doctors often manually draw boundaries on thousands of scan slices, a time-consuming and labor-intensive process.
- **Why Segmentation Matters** – It's the entry point for AI in hospitals, helping with surgery preparation, ER diagnosis, and powering downstream tools like tumor models and 3D printing.
- **What is MedSAM2** – A powerful "Segment Anything" model adapted for 3D medical images and real-time videos. It needs only one box or click to work across modalities.
- **Architecture Upgrades** – Combines the speed of SAM2 with short-term memory blocks for 3D awareness and temporal consistency in video.
- **Training on Massive Multi-Modal Medical Data** – Trained on 450k+ 3D scans and 76k+ echo/endoscopy frames, with human-in-the-loop refinement for accuracy.
- **Code Workflow and Performance** – From CT scans to echo videos, MedSAM2 performs fast, high-accuracy segmentation using a consistent, reproducible PyTorch pipeline.
- **Real Impact** – Up to 90% reduction in manual segmentation time, real-time echo tracking, and generalizability across organs and modalities.

Conclusion

MedSAM2 represents a significant leap forward in medical imaging AI, simplifying 3D and video segmentation with a single prompt, model, and real-time performance. It builds on years of progress in computer vision and applies it directly to the most urgent challenges in healthcare. Faster diagnosis, less manual work, and broader access to precision tools are no longer futuristic goals – they're here now.

See you in the next blog, bye!

References

[MedSAM2: Segment Anything in 3D Medical Images and Videos](https://medsam2.github.io/) (<https://medsam2.github.io/>).

[AI in Medical Imaging Market Size](https://www.globenewswire.com/news-release/2025/02/13/3026027/0/en/AI-in-Medical-Imaging-Market-Size-Projected-to-Reach-USD-14-46-Bn-By-2034.html) (<https://www.globenewswire.com/news-release/2025/02/13/3026027/0/en/AI-in-Medical-Imaging-Market-Size-Projected-to-Reach-USD-14-46-Bn-By-2034.html>).

[Revolutionizing Medical Imaging with Semantic Segmentation](https://keymakr.com/blog/revolutionizing-medical-imaging-with-semantic-segmentation/) (<https://keymakr.com/blog/revolutionizing-medical-imaging-with-semantic-segmentation/>)

[Segment anything in medical images](https://www.nature.com/articles/s41467-024-44824-z) (<https://www.nature.com/articles/s41467-024-44824-z>).